

BME 4503 - Homework #5

This homework set will be fundamental for you to understand how engineers understand and analyze signals, including the biosignals of interest in this course. While the assignment will be worth a few points, I want you to spend a serious amount of time thinking about these questions and their meaning. If you can work through this homework set after a reasonable amount of effort, you will be in good shape for the rest of the course and in all future courses you may take as an engineer.

Here is an overview of what you will accomplish:

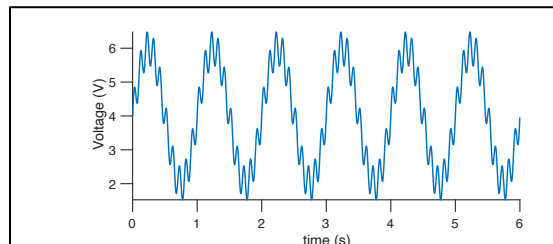
- Plot and represent signals in MATLAB in the time domain AND the frequency domain.
- Manually remove components of a signal in the frequency domain, perform the inverse Fourier transform and compare the result with the original signal.
- Create Bode plots in MATLAB (for magnitude) using known transfer functions of simple filters.
- Show how multiplication in the frequency domain applies the filter to remove certain frequencies in the original signal.
- Explain why the maximum gain for a band pass filter might not be equal to the ideal static gain.
- Be able to use MATLAB effectively to present data and explain concepts.

Introduction Construct the following signal in MATLAB:

$$s = 2 \sin 2\pi t + 0.5 \sin 20\pi t + 4$$

```
>> t=0:1/1000:6-1/1000;  
>> s = 2*sin(2*pi*t) + 0.5*sin(10*2*pi*t) + 4;
```

Plot the signal (show the plot in your submission as figure 1) and verify that it looks like the following:



Let's talk about this signal and get a qualitative description before performing a quantitative analysis. It is composed of two sinusoids and one constant value. The constant, 4 V, acts like an 'offset' for the sinusoids. The sinusoid with the larger amplitude, 2 V, has a frequency of 1 Hz and repeats 6 times over 6 seconds (one oscillation per second = 1 Hz). The other sinusoid has a 10 Hz frequency and smaller amplitude, 0.5 V. It makes the signal look 'fuzzy' or 'noisy'. In fact, we could describe the appearance of our signal by the following:

$$s = 'true\ signal' + 'noise' + 'offset'$$

1.) (1 point) Take the Fast Fourier Transform of the signal, *s*, using the `fft` function in MATLAB.

```
>> S = fft(s);
```

The signal *S* is a representation of *s* in the **frequency domain**. It uses complex numbers keep track of the magnitude and phase of each frequency in the original signal, *s*. The magnitude of the complex number will be related to the amplitude of the original sinusoid. To get the magnitude of the complex signal, *S*, we can just use the `abs` function.

```
>> mag = abs( S/length(S) );
```

Notice that the `length` (number of data points) was used to scale down the magnitude. Since our offset term is a constant, 4 V, we can expect that the magnitude associated with a 0 frequency should be 4. The 0 frequency information is contained in the first element of the data vector, so we can check this assumption by typing the following:

```
>> mag(1)
```

We can look at the entire magnitude vector as a function of frequency using the following as our x-axis:

```
>> fr = (0:length(s)-1) / max(t);
```

and then:

```
>> figure, plot(fr, mag)
>> xlim([-1 11])
```

The `xlim` command allows us to zoom in on the frequencies we care about. You should see a peak at 4 for frequency 0, but the other peaks for 1 Hz and 10 Hz are half of what we would expect. This is because the ‘energy’ is divided across both positive and negative frequency values. In fact, if you replot the figure without the `xlim` command, the negative frequencies will be mirrored at the tail end of the plot (unintuitively appearing to be 990 Hz and 999 Hz magnitude information. This is in fact 1000 Hz -10 Hz or 1000 Hz - 1 Hz). To see this more clearly, you can change the `xlim` limits to [989 1001].

Essentially, this is useless to our analysis, and we will need to account for this to properly show the frequency domain for our original signal. To do so, type the following:

```
>> mag(2:end-1)=2*mag(2:end-1);
```

We have multiplied all magnitudes by 2 except for the first and last components.

Now, plot the following frequency domain representation of our signal and ensure that you properly label all axes (this is figure 2):

```
>> figure, plot(fr(fr<20),mag(fr<20)) % plots only data for 20 Hz or less
>> xlim([-1 21])
```

2.) (1 point) We previously learned to remove offset from a signal by addition or subtraction. Now, we will try to remove the 0 frequency in our frequency domain and then reconstruct a time domain signal that is free of offset. Type the following:

```
>> S(1) = 0;  
>> sfilt = real( ifft(S) );
```

Notice that we are not using the magnitude information, but the original fft result, S, for the signal, s. The ifft function performs the inverse transform of the modified S vector and returns us to the time domain. The signal sfilt might end up being complex because we meddled with S, so we ensure real values by wrapping ifft with the 'real' function ('imag' is its counterpart).

Plot the following to compare sfilt to our original signal, s:

```
>> figure, plot(t, s, t, sfilt)
```

Format and label the figure properly (this is figure 3).

We can now account for offset by filtering out the 0 frequency! Technically, we could also cancel out the 10 Hz signal by subtracting it from the original signal (like noise cancelling headphones), but this is difficult in practice. Can we use what we've learned to remove the 10 Hz component in the frequency domain? YES!

Let's find where the 10 Hz signal is hiding using our magnitude information:

```
>> mag = abs( S/length(S) ); % recalculate mag without the 0 frequency  
>> find( mag>0.2 )
```

The find function looks for the index of components that meet the logical criteria $\text{mag} > 0.2$. You should get `ans = [7 61 5941 5995]` as a result. To validate which of these indices relate to 10 Hz, type:

```
>> fr(ans)
```

Then, to remove the 10 Hz (and 990 Hz) information, type:

```
>> S([61 5941]) = 0;
```

Now, repeat the ifft for this modified S vector and plot the result vs the original signal to show that we were successful in removing the 10 Hz 'noise' (this will be Figure 4).

3.) (1 point) We can move on to consider how the filters we build in the lab help us to remove certain parts of signals (biosignals or otherwise). What we have done so far from part 2 is apply a perfect 'notch' filter to get rid of unwanted frequencies. Real filters are not so perfect, as we shall see.

Recall that the transfer function for an active high pass filter is:

$$H(s) = \frac{V_{out}}{V_{in}} = -\frac{R_2}{R_1} \left(\frac{sR_1C}{1 + sR_1C} \right)$$

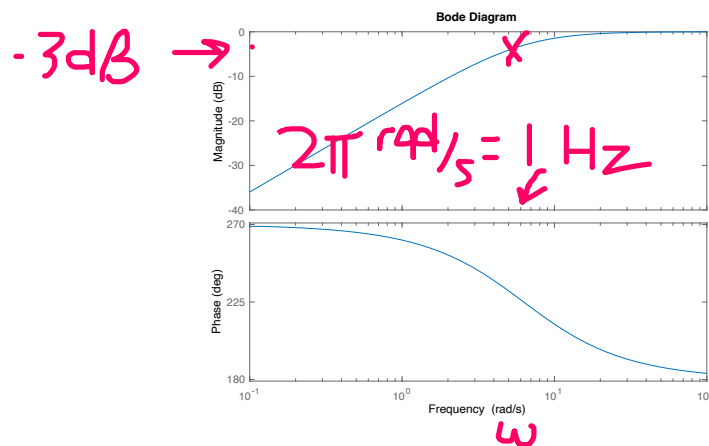
Okay, so you have never seen the terminology $H(s)$ before, but when you read documentation about transfer functions, you will see this convention everywhere. Also note that the Laplace frequency, s , should not be confused for the signal we have defined in MATLAB. For our purposes in lecture (and in the book) we assume $s = j\omega$. To write our transfer function in MATLAB, type:

```
>> H = tf( [-1/(2*pi) 0], [1/(2*pi) 1] )
```

This uses a static gain of 1 and a 1 Hz corner frequency (recall, $f_c = 1/2\pi RC$). The command for the Bode plot is:

```
>> bode(H)
```

Giving us,



The x-axis for the Bode plot is in terms of ω and it uses the logarithmic scale. Our corner frequency of 1 Hz will be at $\omega = 2\pi$ rad/s (drawn in pink). This value corresponds to -3 dB, which means that the output magnitude is about 70% of the input magnitude at this corner frequency. The formula for decibels (β) is:

$$\text{Gain } (\beta) = 20 \log_{10} \left(\frac{V_{out}}{V_{in}} \right)$$

We can get the gain information by specifying an output from the bode function:

```
>> m = bode( H, 2*pi*fr ); % gets gain for the frequencies in the fr
vector (converted to rad/s)
>> m = m(:); % converts to a row vector
>> m(3002:end) = m(3000:-1:2); % mirrors the filter
```

Strangely, the output, m , is in terms of V_{out}/V_{in} so if you want to recreate the first plot, you might try:

```
>> figure, loglog( fr, 20*log10(m) ), set(gca, 'yscale', 'linear')
```

The m vector is convenient though, because we can just perform an element-wise multiply between S and m' (transposed to make a row vector) to filter our signal. Once we take the ifft of the modified S vector, we can inspect the new signal in the time domain. Type the following:

```
>> S=fft(s);  
>> sfilt3=ifft(S.*m');
```

Plot a comparison between `sfilt3` and the original signal, `s`, to show that the 0 frequency has been effectively removed (this will be figure 5 in your submission). Since the corner frequency was at 1 Hz, a portion of the 1 Hz frequency was removed (it is at 70% of the original magnitude), and nearly all of the 10 Hz signal has ‘passed’.

4.) (1 point) Derive the transfer function for an **active low pass filter** with a 1 Hz corner frequency and **unity gain** (gain = 1). Create a Bode plot for this filter’s transfer function and then apply the filter to our original signal, `s`. Plot a comparison between the original signal and the filtered result, properly labeling all signals and axes. These plots will be figures 6 & 7 in your submission.

5.) (1 point) Derive the transfer function for an **active band pass filter** with corner frequencies at 500 mHz and 2 Hz and **unity gain** (gain = 1). Create a Bode plot for this filter’s transfer function and then apply the filter to our original signal, `s`. Plot a comparison between the original signal and the filtered result, properly labeling all signals and axes. These plots will be figures 8 & 9 in your submission.

You might expect the amplitude of the 1 Hz signal to remain unchanged, but in reality it is too near the corners to remain unaffected. While the ideal static gain would be equal to 1, the actual max gain can be found by typing the following:

```
>> m=bode(H,2*pi*fr);  
>> m=m(:); max(m)
```

Does this value reflect what you see in your plot of the filtered result?